Face-off with *Sizeof()*

In C programming, the unary operator *sizeof()* returns the size of its operand in bytes. The *sizeof()* operator is discussed in detail in this article, along with illustrative examples of code.

Sizeof() is used extensively in the C programming language. It is a compile-time unary operator, which can be used to compute the size (in bytes) of any data type of its operands. The operand may be an actual type-specifier (such as *char* or *ints*) or any valid expression. The *sizeof()* operator returns the total memory allocated for a particular object in terms of bytes. The resultant type of the *sizeof()* operator is *size_t*. *Sizeof()* can be applied both for primitive data types (such as *char, ints, floats*, etc) including pointers and also for the compound data types (such as structures, unions, etc).

Usage

The *sizeof()* operator can be used in two different cases depending upon the operand types. Let me elaborate this with syntax, examples and sample codes.

Case 1: When the operand is a type-name

When *sizeof()* is used with 'type-name' as the operand (such as *char, int, float, double*, etc), it returns the amount of memory that will be used by an object of that type. In this case, the operand should be enclosed within parenthesis. The *sizeof* operator is used when the operand is a type-name as shown in Table 1.

Illustration: The sample demo code is shown below (Code 1), and the output of the code when I run it in my system is shown in Figure 1.

Code 1

```
 1 #include <stdio.h>
 2 #include <stddef.h>
 3
 4 int main()
 5 {
 6         printf("Sizeof(char) : %lu\n", (long unsigned )
sizeof(char));
 7         printf("Sizeof(int) : %lu\n", (long unsigned )
sizeof(int));
 8         printf("Sizeof(float) : %lu\n", (long unsigned )
sizeof(float));
 9         printf("Sizeof(double) : %lu\n", (long unsigned )
sizeof(double));
```